

# 协议转换课题：执行方案与问题讲解

犀牛鸟开源实战 · TRACK 05A / 05B 协议转换组

基于《协议转换课题\_问题全景分析》整理 · 面向无经验队员 · 三周 · 截止 9 月 14 日

四项决策已拍板

OpenAI Chat ↔ Anthropic

OpenAI Response ↔ Anthropic

## 一、这个课题到底在干什么

你的团队要在三周内做一个「协议转换器」。背景是：OpenAI 和 Anthropic 两家大模型的接口「语言」不一样，但现在很多程序是照着 OpenAI 的格式写的，想换成 Anthropic 的模型跑，或者反过来。你的任务就是写一个**翻译层**，让两边能互相听懂。

**核心论点：**真正难的从来不是字段翻译，而是两家的「底层设计哲学」不一样——就像翻译两句话容易，翻译两种思维方式很难。整份方案都在回答一个问题：**难在哪、怎么避免三周白干。**

**硬件说明：**本课题**不需要任何 GPU**（含 A40）。所有模型推理都发生在 OpenAI / Anthropic 的云端，本地只做协议转换（HTTP 转发、JSON 字段映射、SSE 透传），普通笔记本 CPU 足够。真正的成本来源是 **API 调用费（token 计费）**，不是显卡。

### 1.1 全篇的钥匙：IR（中间表示）

方案开篇说了一句最关键的话：**这两个课题不是两个平行任务，而是一个 IR 的两个适配器。**

- **如果没有 IR：**三个协议两两互转，要写 6 个方向的转换器，而且 Chat 和 Response 之间根本没法直接互转（结构差异太大）。
- **有了 IR：**先定义一个「内部通用格式」，每个协议只写「转成 IR」和「从 IR 转出」两个方向，三个协议 = **3 对 adapter**，工作量从 6 压到 3。
- **IR 是两队人马之间的唯一契约：**它一冻结，05A 和 05B 就能并行开发，第三周才能合得起来。

所以第一周最重要的事不是写代码，而是**冻结 IR v0**。IR 分三层：L0（内容块模型）、L1（规范请求）、L2（会话与缓存上下文）。

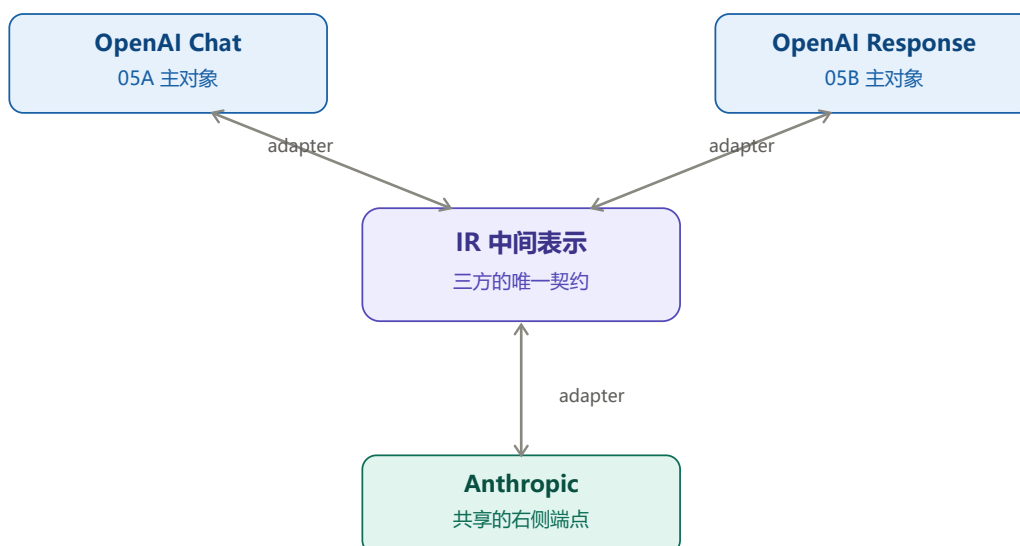


图 1 · IR 星型结构：把 6 个方向的转换压成 3 对 adapter

## 1.2 难点一：有状态 vs 无状态的根本冲突

OpenAI Response 协议有个字段叫 `previous_response_id`，意思是「上一轮内容我已存在服务端，你下一轮只传这个 ID 就行」——这是省 token 的设计。但 Chat 和 Anthropic 是「记性差」的协议，要求你**每一轮都把完整历史重发一遍**。

**核心矛盾：**把 Response 的请求转给 Anthropic 时，服务端根本没有你上一轮的东西，`previous_response_id` 就「无处安放」。结论是：转换层不能只是「翻译函数」，必须**自己变成一个带存储的服务**（存会话、处理并发分叉、处理过期淘汰）。这直接决定了整个项目的工程量。

## 1.3 难点二与三：记忆注入 × KV Cache（课题核心交付）

先讲清两个词：

- **记忆注入：**把「关于用户的长期记忆」塞进每次请求的提示词，让模型每次都「记得」。
- **KV Cache（缓存）：**Anthropic 为省钱，会把请求开头的相同部分缓存起来。两轮前缀**一模一样**，第二轮就能省 90% 费用。

**它们撞在一起的地方：**你每轮注入记忆，若记忆的内容、顺序、粒度稍有变化，前缀就变了 → 缓存全失效 → 每轮重新付费。

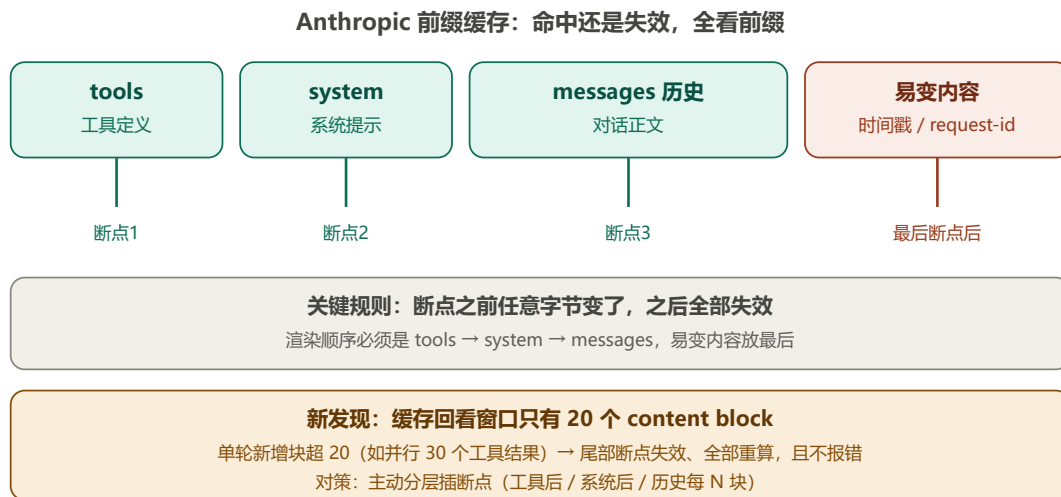


图 2 · 前缀缓存与断点机制

## 二、最精华的洞察（最值得记住的 6 条）

| 洞察                | 为什么优秀                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------|
| 1. 产出不是转换器，而是量化文档 | 「一份说清楚三协议在记忆注入与缓存语义上差在哪、损失多大、怎么取舍的量化文档」——这个定位极其准确。转换器是载体，量化取舍才是别人拿不走的东西，正好对应结业标准的「影响可解释」。    |
| 2. IR 分层 = 缓存断点布局 | 把 IR 的 L1/L2 分层直接变成「工具后 / 系统后 / 历史每 N 块」的断点策略，一个设计同时解决架构与缓存两个问题，是全篇最漂亮的一次贯通。                 |
| 3. 20-block 回看窗口  | 本次核对中新发现、原文档完全没写的硬约束。让实验数据失真却不报错的隐形杀手，是被验证过的最有价值增量。                                          |
| 4. 缓存阈值非单调        | Opus 代次之间阈值不单调（4.5/4.6 是 4096，4.7 降到 2048，4.8 回 1024，5 才是 512），不能靠外推，必须逐模型查表——反直觉但已逐条核对官方文档。 |
| 5. 命中率可观测才能可信     | 「没有命中率可观测，结论不可信」——方法论上的自觉，把「可解释」从口号变成了硬指标。                                                   |
| 6. 盈亏平衡精确计算       | 5 分钟档命中 1 次回本、1 小时档需命中 2 次；不做系数就跨协议对比成本，结论必错。                                                |

## 三、执行方案

### 3.1 一句话方案（北极星）

**用数据回答一个问题：**在 OpenAI Chat / Response 与 Anthropic 之间做记忆注入，缓存命中的损失到底是多少？哪种注入策略最省钱？

转换器只是「实验装置」，最终交付物是一份**量化取舍文档**。

### 3.2 最终决策（四项已拍板）

| 决策                   | 最终选择                  | 核心理由                                   |
|----------------------|-----------------------|----------------------------------------|
| Session init 弹网页     | 纳入，做 B 类上下文编排         | 创新点；网页展示「注入了什么 + 缓存前缀」，可勾选裁剪；与状态层一石二鸟  |
| PR 落点                | 自开仓库                  | 节奏自己掌控，9/14 前最稳；代码按可移植标准写，之后再提上游       |
| previous_response_id | 必须支持                  | 课题唯一「有状态」的点，是状态层的自然产物                  |
| 转换层落点                | 网关/代理（独立进程） + 独立状态层模块 | 见下方推理链条：被「必须支持 previous_response_id」锁死 |

#### 为什么转换层落点锁定「网关/代理」

previous\_response\_id 必须支持

↓ 转换层必须是有状态服务

SDK shim（无状态进程内库）→ 出局

↓ 只剩网关/代理能承载状态存储

harness 内 adapter → 绑定具体 harness，与「自开仓库」冲突 → 出局

↓

结论：网关/代理（独立进程）+ 独立状态层模块

工作量控制三点：① 状态存储做成独立模块，先做最小可用版（会话表 + TTL 淘汰）；② 鉴权/限流/重试先从简；③ 与弹网页方案共用同一套状态层，不重复建设。

### 3.3 北极星指标

**缓存命中率（cache\_read 命中率）**，辅以「总 token 成本」和「被丢弃的参数清单」。

- 判定缓存是否命中：看 usage 里 cache\_creation\_input\_tokens 与 cache\_read\_input\_tokens，两者同时为 0 = 未缓存。
- 第一天就搭好命中率埋点**，而不是最后补。没有这个数字，任何结论都不可信。

3.4 三周路线

| 周次               | 核心任务                                        | 关键产出                                    |
|------------------|---------------------------------------------|-----------------------------------------|
| WEEK 01<br>冻结契约  | 冻结 IR v0；搭命中率埋点；跑通单轮文本链路；能力矩阵 v0；定状态层最小方案   | ir-schema.md（共同签字）；端到端 demo；token/缓存对照表 |
| WEEK 02<br>打通主链路 | 三对 adapter 多轮 + 工具调用；思考块往返；记忆注入 + 幂等去重；分层断点 | 工具 ID 映射表； <b>开始采集命中率数据</b>             |
| WEEK 03<br>量化交付  | 跑对照实验；复现 20-block 断崖；往返测试；PR 整理 + 量化文档      | 实验结论 + 失败案例；有损字段清单；最终文档                 |

3.5 杀手铜实验设计（三维度对照）

把「记忆注入 × 缓存命中率」这个课题核心问题**用数据钉死**。记忆注入有三个**互相独立**的维度，之前容易混在一起，这里拆开分别做对照。

| 先分清三个独立维度 |                  |                                           |
|-----------|------------------|-------------------------------------------|
| 维度        | 含义               | 对应的文档硬约束                                  |
| 位置        | 记忆放在前缀头部，还是消息流尾部 | 前缀稳定性、previous_response_id「改写前缀」冲突        |
| 粒度        | 记忆块多大、拆成几条       | 最小可缓存阈值（512/1024/2048/4096）、20-block 回看窗口 |
| 更新频率      | 多久变一次（锁死 vs 每轮变） | 前缀稳定性、缓存失效                                |

实验一 • 更新频率 × 命中率

| 组     | 策略                  | 预期                  |
|-------|---------------------|---------------------|
| 对照组 A | 不注入记忆（基线）           | 命中率最高，但模型记不住用户偏好    |
| 组 B   | 会话开始注入一次并锁死（确定性序列化） | 命中率高，是推荐策略的候选       |
| 组 C   | 每轮动态注入（内容/顺序/频率变化）  | 命中率断崖下跌，证明「为什么必须锁死」 |

实验二 • 位置 × 命中率

| 组    | 策略                           | 预期                                     |
|------|------------------------------|----------------------------------------|
| 组 D1 | 记忆放前缀头部（system 之后）           | 命中稳定，但占用缓存前缀                           |
| 组 D2 | 记忆放消息流尾部（Responses 侧 append） | 展示与 previous_response_id 的冲突，模型注意力位置改变 |

实验三 • 粒度 × 命中率（本次新增，直接验证两条硬约束）

粒度是之前遗漏的维度。它直接命中文档第 10、11 条两条硬约束——块太小会静默不缓存，块太碎会触发 20-block 回看窗口失效。

| 组    | 粒度设置                     | 验证的硬约束        | 预期                                      |
|------|--------------------------|---------------|-----------------------------------------|
| 组 G1 | 单块约 300 token（低于 512 阈值） | 最小可缓存长度       | 静默不缓存：cache_read 与 cache_creation 同时为 0 |
| 组 G2 | 4 条 × 512 token 小块       | 阈值边界 + 多块     | 可缓存，但占用 4 个 content block               |
| 组 G3 | 单块 2048 token 大块         | 超阈值稳定命中       | 稳定命中，token 成本最高                         |
| 组 G4 | 30 条细粒度条目                | 20-block 回看窗口 | 单轮新增 >20 block，尾部断点失效、全部重算              |

衡量指标：cache\_read 命中率（核心）、每轮总 token 成本（含 1.25x / 2x 写入溢价，5 分钟档 vs 1 小时档分开算）、盈亏平衡点（5 分钟档命中 1 次回本，1 小时档需命中 2 次）。

**期望结论（提前写好的假设）：**① 更新频率——只有「会话内锁死 + 确定性序列化」才不伤缓存，每轮动态注入让命中率从 X% 掉到接近 0 且不报错。② 位置——头部注入稳定，尾部 append 与 previous\_response\_id 冲突。③ 粒度——块太短（低于最小可缓存阈值）静默不缓存，块太碎（单轮超 20 block）触发回看窗口断崖，**存在一个最优粒度区间，需逐模型查阈值表。**

3.6 明确不做（写进文档的已知限制）

- 多模态（image / document / audio）转换
- Realtime（WebSocket）与 Batches 端点
- Bedrock Converse / Vertex 信封适配
- 内置工具的通用映射（只给拒绝清单）
- MCP 工具语义保真（降级为 function）
- 结构化输出约束保证（显式降级策略 + 告警）
- Gemini / 国产模型方言兼容

3.7 剩余待办（唯一还需要外部输入的）

| 待办         | 说明                                          | 动作                     |
|------------|---------------------------------------------|------------------------|
| 记忆注入策略由谁定义 | 需要 TRACK 01 给出「注入什么内容、多大粒度、多久更新」，缓存实验才有稳定输入 | 主动去找 TRACK 01 要这份输入，别等 |

**给无经验队员的最后建议：**四项核心决策已定、逻辑自治——有状态服务是主线，网关承载它，状态层模块同时喂给 previous\_response\_id 和弹网页方案，自开仓库保证 9/14 节奏。剩下唯一要推进的是**去找 TRACK 01 要记忆注入策略的输入**，这一步明天开分工会前就做，这周不用返工。